

CSE 5311 — Sorting Project Demo Prep

Arham Shams Sameer 1002078834

April 27, 2026

Algorithm Complexity Cheat Sheet

Algorithm	Best	Average	Worst	Space	Stable?	In-place?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Yes
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No	Yes
Quick Sort (regular)	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	Yes
Quick Sort (med-3)	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	Yes

Stability = elements with equal keys keep their original relative order. Selection sort is unstable because a distant swap can jump over equal elements. Heap sort is unstable because heapify rearranges equal-key elements arbitrarily.

Algorithm-by-Algorithm Walkthrough

Bubble Sort

How it works: Repeatedly walk the array comparing adjacent elements, swapping if out of order. After each pass the largest unsorted element “bubbles up.”

Optimization – early exit with swapped flag:

- First version (commented out) always ran all $n-1$ passes $\Rightarrow O(n^2)$ even on sorted input.
- The **swapped** flag detects “no swaps this pass” $\Rightarrow$ break early $\Rightarrow O(n)$ best case.
- “On sorted input, the first pass does zero swaps, so it breaks immediately.”

Selection Sort

How it works: Find the minimum in the unsorted portion, swap it into position i .

Key details:

- `if min_idx != i` avoids a useless self-swap.
- Always $O(n^2)$ even best-case — always scans the entire unsorted portion.
- Unstable: $[5_a, 3, 5_b, 2]$ — swapping 5_a with 2 puts 5_a after 5_b .

Insertion Sort

How it works: Take element at position i , slide it left into the correct spot in the sorted prefix.

- $O(n)$ best-case: if input is sorted, the **while** loop never executes.
- The **key** variable stores `a[i]` before shifting so the value isn’t lost.
- **Adaptive:** runtime is $O(n + \text{inversions})$.

Merge Sort

How it works: Divide the array in half recursively until size 1, then merge back up in sorted order.

Why I switched from the top-down version (commented out):

- Old version created new list slices (`arr[:mid]`, `arr[mid:]`) at every level — excessive copying.
- Final version operates on a single array with index bounds; only copies during the merge step.

Stability: The merge step uses `<=` (not `<`), so equal elements from the left half go first $\Rightarrow$ stable.

Space: $O(n)$ for the temporary `left_half` / `right_half` arrays.

Recurrence: $T(n) = 2T(n/2) + O(n)$. Master Theorem case 2 $\Rightarrow O(n \log n)$.

Heap Sort

How it works: Build a max-heap, then repeatedly extract the max (swap root to end), shrink heap by 1, re-heapify.

Why I switched from top-down to bottom-up heap construction (commented out):

- Top-down (sift-up): insert one by one $\Rightarrow O(n \log n)$ to build.
- Bottom-up (sift-down, Floyd's): start from last internal node ($n//2 - 1$) $\Rightarrow O(n)$ to build. Most nodes are near the leaves and only sift a small distance.

Children formulas (0-indexed): Left = $2i + 1$, Right = $2i + 2$.

Quick Sort — Regular (Lomuto, last-element pivot)

How it works: Pick pivot, partition so everything $\leq$ pivot is left, $>$ pivot is right, recurse.

Why last-element over first-element (commented out): “First-element partition going right-to-left was harder to follow. Last-element with Lomuto is more intuitive — scan left to right, swap small elements to the front.”

Lomuto partition walkthrough:

1. `pivot = a[high]`
2. `i = low - 1` — boundary of the $\leq$ pivot section
3. For each `j` from `low` to `high-1`: if `a[j] <= pivot`, increment `i`, swap
4. Swap pivot into `i+1`, return `i+1`

Worst case $O(n^2)$: Pivot is always min or max (sorted/reverse input). Each partition removes only 1 element $\Rightarrow n$ levels $\times n$ work.

`sys.setrecursionlimit(200000)`: Python's default is 1000. With 10K elements on sorted input, quick-sort can recurse $\sim 10K$ deep $\Rightarrow$ stack overflow without this.

Recurrence: Average $T(n) = 2T(n/2) + O(n) \Rightarrow O(n \log n)$. Worst $T(n) = T(n-1) + O(n) \Rightarrow O(n^2)$.

Quick Sort — Median-of-Three

Same as regular, but picks the median of `a[low]`, `a[mid]`, `a[high]` as pivot.

Bug I fixed (commented out): First version returned the median *value* instead of the *index*. “I was returning `triple[1]` which is the value, but I needed the index so I could swap the median into the pivot position.”

Why it helps: On sorted input, the median of first/middle/last is roughly the true median $\Rightarrow$ balanced partition $\Rightarrow O(n \log n)$ instead of $O(n^2)$.

`_partition_median3` swaps the median into `high`, then reuses `_partition_last` — no code duplication.

Code Design Decisions

Why does every function return a new list?

I copy the input with `a = list(arr)` so the original is never modified. This makes benchmarking fair — each algorithm gets the exact same input — and makes correctness testing easy with `assert result == sorted(data)`.

Why `time.perf_counter()` instead of `time.time()`?

`time.time()` has ~ 15 ms resolution on Windows. Fast algorithms on small inputs showed 0.0 seconds. `perf_counter()` uses the highest-resolution clock available — typically sub-microsecond.

Why a nested dict for results instead of a flat list?

First version stored `(dist, name, size, elapsed)` tuples — annoying to query for charts. The nested dict `results[dist][name][size]` gives $O(1)$ lookup.

Why `matplotlib.use('Agg')` in benchmark but `'TkAgg'` in GUI?

`Agg` is non-interactive, renders to PNG files — no window needed. The GUI needs `TkAgg` to embed a live chart inside Tkinter.

Why threading in the GUI?

First version ran the benchmark on the main thread — froze the entire window. A `daemon` thread keeps the UI responsive. `self.after(0, ...)` pushes results back to the main thread safely (Tkinter isn't thread-safe).

Why daemon thread?

If the user closes the window during a long sort, a daemon thread dies automatically. Without it, the program hangs.

Why a dict as the algorithm registry (ALGORITHMS)?

Maps names to function references. Benchmark and GUI both import it. Adding a new algorithm is a one-line change — no if-else chains.

Why `seed=42` everywhere?

Reproducibility. Same size $\Rightarrow$ same array $\Rightarrow$ comparable benchmark results across runs.

Benchmarking & Results

Why these sizes (100–10,000)?

Small enough that $O(n^2)$ finishes in seconds, large enough to see the scaling gap vs. $O(n \log n)$.

Why these 4 distributions?

- **Random:** Average-case for all algorithms.
- **Sorted:** Best-case for insertion/bubble, *worst-case for regular quicksort*.
- **Reverse:** Worst-case for bubble (max swaps) and regular quicksort.
- **Nearly sorted:** Realistic scenario — insertion sort shines ($\approx O(n)$).

Key Chart Observations

1. $O(n^2)$ vs $O(n \log n)$ split: bubble/selection/insertion form a clear upper group; merge/heap/quicksort form the lower group.
2. Insertion sort on sorted/nearly-sorted: nearly flat, close to $O(n)$, faster than $O(n \log n)$ algorithms.
3. Bubble sort on sorted input: also $\approx O(n)$ thanks to the early-exit flag.
4. Regular quicksort on sorted/reverse: dramatic spike — the $O(n^2)$ worst case.
5. Median-of-three quicksort stays $O(n \log n)$ on sorted input (comparison chart shows this).
6. Heap sort: consistent across all distributions (heapify doesn't care about input order).
7. Selection sort on sorted input: still $O(n^2)$ — no early exit.

Which algorithm was fastest overall?

On random input, quicksort is typically fastest in practice due to cache locality and low constant factors. On sorted input, insertion sort wins at $O(n)$.

Input Generator & Self-Test

How does `generate_nearly_sorted` work?

Start with a sorted array, do $n/20$ random adjacent swaps ($\sim 5\%$ of elements displaced).

Why one integer per line instead of JSON?

First version used JSON (commented out). Plain text is simpler to debug and `load_input` is a one-liner list comprehension.

How do you verify correctness?

Two layers: (1) Self-test in `sorting_algorithms.py`: all 7 algorithms $\times$ 8 edge cases, asserts `result == sorted(case)`. (2) Benchmark assertion: `assert result == sorted(data)` after every run.

What are the 8 edge cases?

Empty `[]`, single `[1]`, two elements `[2,1]`, general unsorted, already sorted, reverse sorted, 100 random ints, all duplicates `[7]*50`.

GUI Questions

Walk through what happens when “Run Benchmark” is clicked.

1. Validate: ≥ 1 algorithm selected, valid integer size. Warn if $O(n^2)$ with $n > 50000$.
2. Disable Run button, clear old results, update status bar.
3. Spawn daemon thread: generate input, run each algorithm with `perf_counter`, push each result to UI via `self.after(0, _append_row)`.
4. `_finish_run`: redraw bar chart, label bars, show fastest in status bar.
5. Re-enable Run button.

Why CheckButtons over RadioButtons?

Radio buttons only allow one selection (commented out). Comparison requires selecting multiple algorithms side-by-side.

Why Treeview over Listbox?

Listbox is a flat list of strings. Treeview gives proper table columns (algorithm, time in s, time in ms).

What about the animation code?

Attempted using `FuncAnimation` inside Tkinter (commented out). Didn't work — `FuncAnimation` conflicts with Tkinter's event loop, bars wouldn't update. Left the code to show the attempt.

Curveball Questions

What is the recurrence relation for merge sort?

$T(n) = 2T(n/2) + O(n)$. Master Theorem: $a=2, b=2, f(n)=n$. $\log_b a = 1$, $f(n) = \Theta(n^1) \Rightarrow$ Case 2 $\Rightarrow T(n) = O(n \log n)$.

Can you make quicksort $O(n \log n)$ worst-case?

Need deterministic median-of-medians (BFPRT) for $O(n)$ median selection, guaranteeing balanced partitions. But the constant factor is large — randomized/median-of-three is better in practice.

Why not use Python's built-in sort?

Project requires implementing from scratch. Python's sort is Timsort — a hybrid of merge sort and insertion sort. $O(n \log n)$ worst, $O(n)$ best, stable.

Could you improve your quicksort further?

- Switch to insertion sort for small subarrays ($n < 10-20$).
- Tail-call optimization: recurse on smaller partition, iterate on larger $\Rightarrow O(\log n)$ stack.
- Introsort: switch to heapsort if depth $> 2 \log n \Rightarrow O(n \log n)$ worst-case guaranteed.

Lower bound for comparison-based sorting?

$\Omega(n \log n)$. Decision tree model: $n!$ permutations, each comparison splits into 2 branches $\Rightarrow$ tree height $\geq \log_2(n!) \approx n \log n$.

What's an inversion?

A pair (i, j) where $i < j$ but $a[i] > a[j]$. Insertion sort does one swap per inversion $\Rightarrow O(n + \text{inversions})$.

What if I give you negative numbers or duplicates?

Works fine. Algorithms compare with $<$, $>$, $<=$ — valid for negatives. Duplicates tested with `[7]*50`.

Code Quality Highlights

- Version history in every file header with dates
- Commented-out previous attempts show iterative development
- Comments explain *why*, not *what*
- Correctness assertions at every level (self-test + benchmark)
- Modular design: algorithms, input generation, benchmarking, GUI are separate files
- Single source of truth: `ALGORITHMS` dict defined once, imported everywhere
- No external dependencies beyond matplotlib — everything else is `stdlib`

Suggested Demo Flow (15 minutes)

Time	Action
0–2 min	Run <code>run.bat</code> , show self-test passing
2–4 min	Show input files generated, explain 4 distributions
4–7 min	Benchmark running — point out $O(n^2)$ vs $O(n \log n)$ speed gap in real-time
7–10 min	Open charts — explain key observations
10–13 min	GUI demo: comparison, change distribution to sorted, show QS degradation
13–15 min	Q&A

Tip: If the benchmark is running long during the demo, skip to the GUI with $n=2000$. Show the pre-generated charts from `results/` for full benchmark data.